QUICK START · V0.2.9 · OPEN SOURCE (MIT)

Give your agent *a run button.*

Baton lets you and your coding agent run, hot-reload, read logs from and screenshot your apps: Flutter, Next.js and Vite, React Native and Expo, native iOS and Android. One MCP server, a CLI and a local daemon. Node.js 24+, on macOS, Linux and Windows.

1 Install, then add Baton to every agent in one command

```
$ npm install -g baton-run
$ baton setup
+ Claude Code          added · plugin
+ Codex                added ~/.codex/config.toml
+ Cursor               added ~/.cursor/mcp.json
  Restart those apps (or start a new chat) to load Baton.
```

`baton setup` finds the agents installed on this machine and adds Baton to each: **Claude Code** (as the plugin), **Codex**, **Cursor**, **VS Code**, **Antigravity**, **Gemini CLI** and **Windsurf**. It edits only the `baton` entry and backs up each file it changes.

`baton setup --dry-run`	Show what it would change, change nothing
`baton setup cursor codex`	Only the agents you name: `claude codex cursor vscode antigravity gemini windsurf`
`baton setup --remove`	Take Baton out again
`npx -y --package=baton-run baton setup`	The same, without a global install
`npx -y baton-run`	No install: this command starts the MCP server, for any agent's config

2 Ask your agent to run the app

Open a Flutter, web, React Native, Xcode or Gradle project and say *"run the app on an iPhone simulator and show me a screenshot"*. Baton reads what is already there (`.vscode/launch.json`, `package.json`, `pubspec.yaml`, Xcode schemes, Gradle modules). The agent's loop:

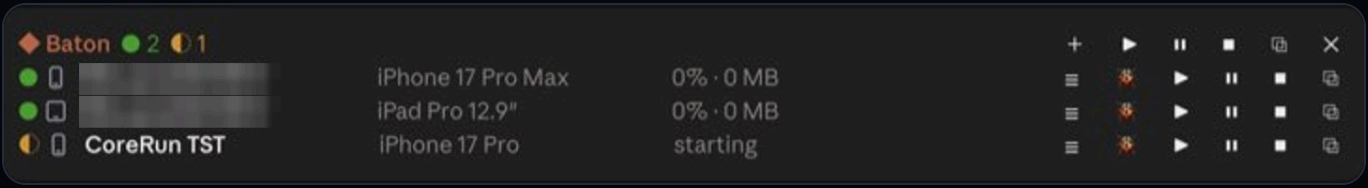
`inspect_project` → `boot_device` → `run_target` → `wait_for` → `edit` → `check_change` ↻

`check_change` applies each edit (hot reload for Flutter, HMR for web and React Native, restart for native) and answers OK or NOT OK with only what is new: errors, failed requests and a screenshot. The agent fixes what it reports and checks again.

IN CLAUDE CODE

Every run, *one row of buttons.*

With the plugin installed, a Baton band sits above the prompt while anything runs, in the terminal and the desktop app. It reads the same daemon as the tools, so runs started by Claude, the `baton` CLI or the Baton app all show up.



The band in the Claude desktop app: two private app runs (names blurred) on an iPhone 17 Pro Max and an iPad Pro, and CoreRun, a habit and streak tracker, still starting.

On each row

- ☰ **Logs** open under the row
- 🐞 **Debug**: Claude reads the run's errors and logs, fixes the cause, then runs `check_change`
- ▶ **Play**: hot reload, resume a paused run, or run it again once it ends
- ⏸ **Pause** (macOS; not Android emulators)
- ■ **Stop**. An ended run shows ✕ to dismiss it
- ⧉ **Bring to front** raises that run's own simulator window, so the iPad comes forward, not whichever window was last. A web run opens its URL (macOS)

In the header

- `+` **New run**: puts `/baton:run` in the prompt
- ▶ ⏸ ■ ⧉ The same, for every run at once
- ✕ Hide the band until `/baton`

`/baton` opens the same view in a side pane with longer logs. `/baton:run` and `/baton:doctor` come with it.

Prompts to try

```
› run the app on an iPhone and an iPad simulator and show me both screenshots
› change the header colour, check_change, and tell me what you see
```

Something off? `baton doctor` **or** `/baton:doctor`

github.com/kanumuri9593/Baton · kanumuri9593.github.io/Baton · Free and open source (MIT) · Built by Yeswanth Varma Kanumuri